Sistemas Operativos

Ayudantía 2: Threads y Scheduling

Profesor: Víctor Reyes
Ayudante: Diego Banda



Contacto

diego.banda@mail.udp.cl

darklouds

Click aquí!!



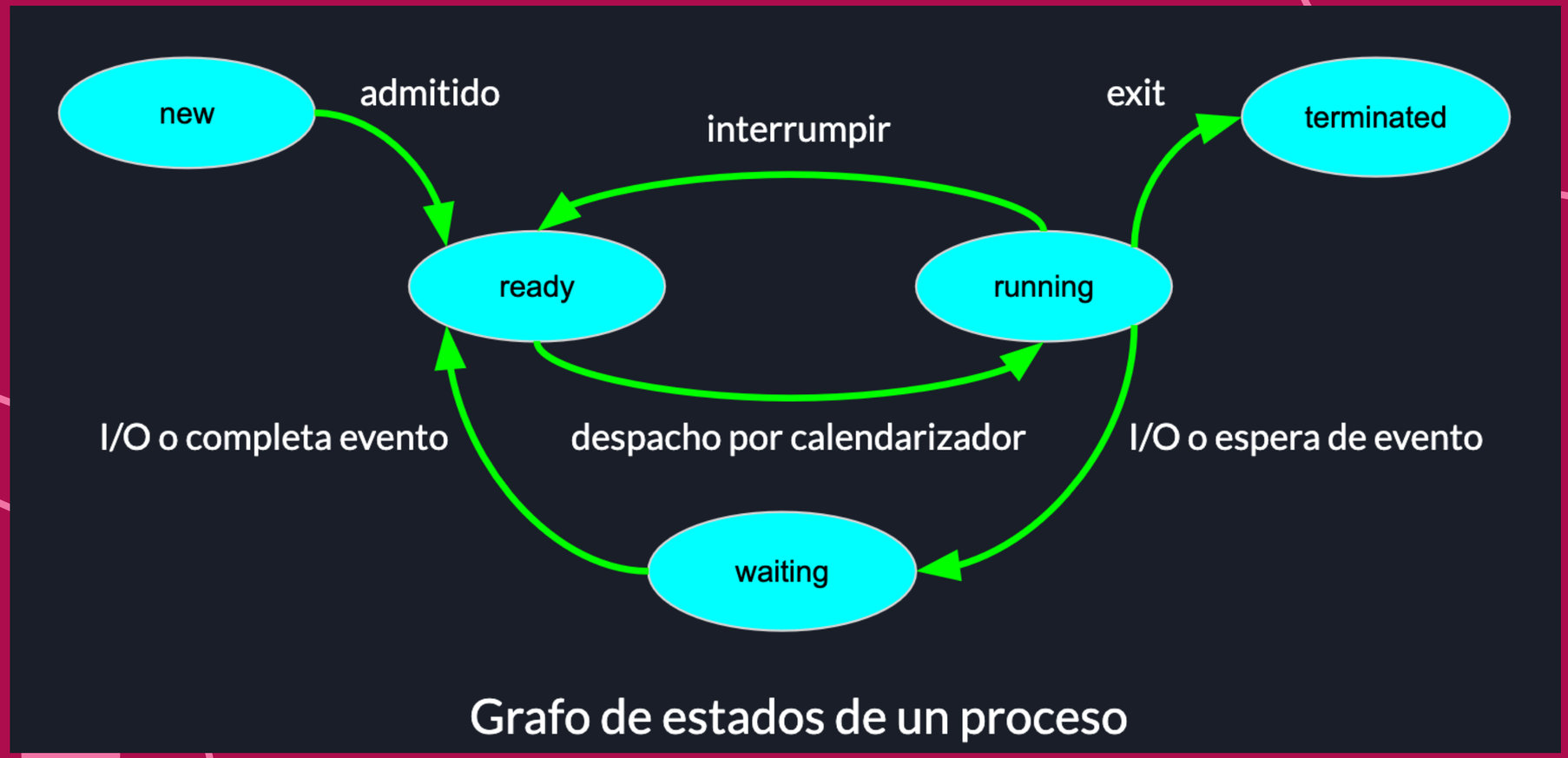
Scheduling

Imaginando que se tiene un procesador de un solo núcleo...
Se necesita que cada proceso actual tenga tiempo de CPU.

Si tu PC tiene más programas que núcleos el procesador, ¿cómo parece ejecutar todo simultáneamente?



Scheduling recordando...



Grafo de estados de un proceso

Scheduling

Long term scheduler

Procesos que quedan en la ready queue, esto define la cantidad también.

Medium term scheduler

Modifica temporalmente el grado de multi-programación y hace swapping (RAM a HDD y viceversa).

Short term scheduler

Envía procesos de la ready queue a CPU, realiza cambio de contexto.



Tipos de scheduling

Scheduling expropiativo: Puede quitar un proceso que está con tiempo de CPU para darle tiempo de CPU a otro.

Scheduling no-expropiativo: Una vez asignó tiempo de CPU a un proceso, este debe terminar para seguir con otro.

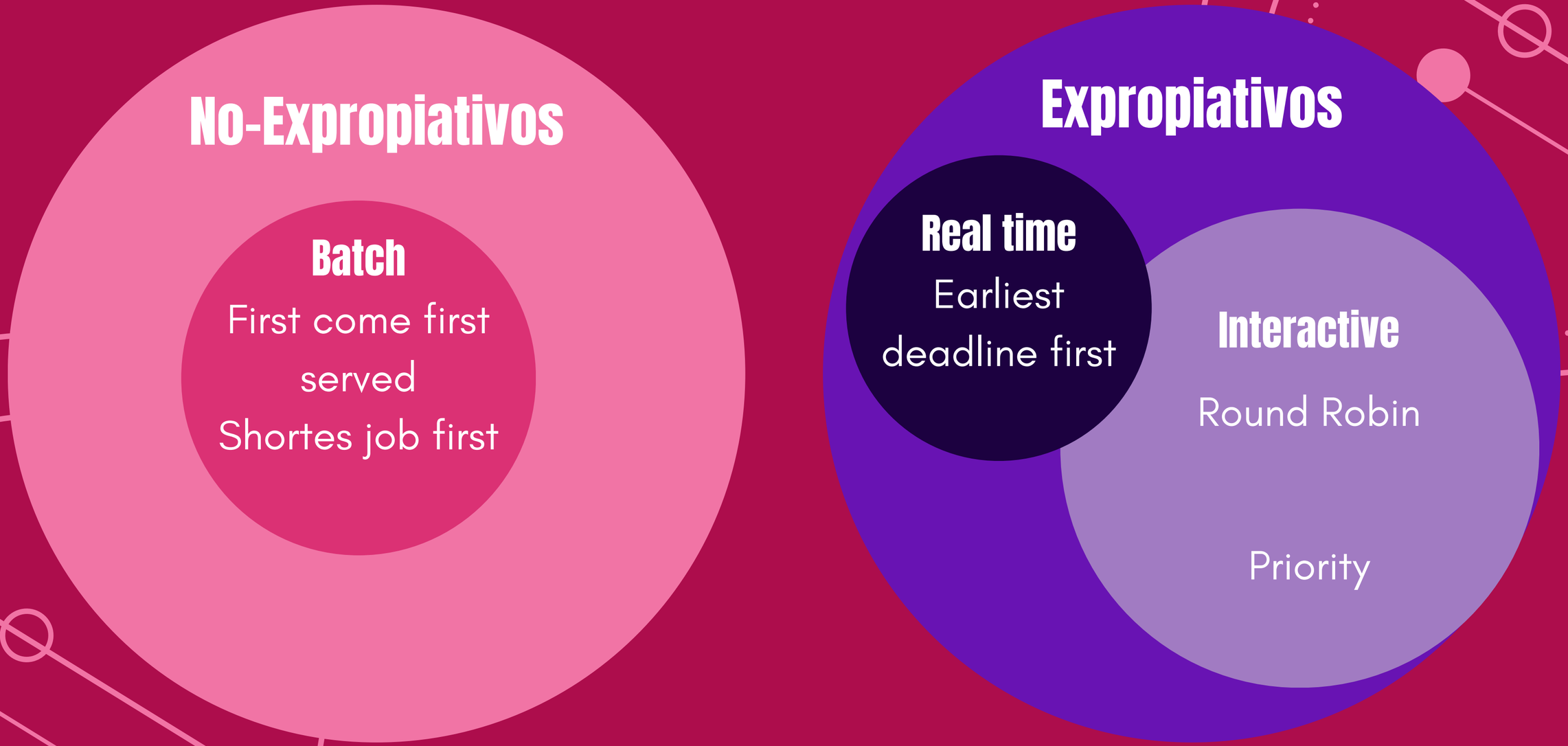
Batch Scheduling: Se ejecutan bajo cierta secuencia.

Real-time Scheduling: Responder ante tiempos de término es crítico.

Interactive Scheduling: Principalmente para dar respuesta rápida entre máquina/usuario y procesos paralelos.



Algoritmos de scheduling



Ejercicios Scheduling

Considere la siguiente tabla de proceso:

Proceso	CPU Burst	Arrival Time
P_0	3	7
P_1	10	1
P_2	3	0
P_3	6	5

Determinar el Turnaround y end time, considerando para ello el algoritmo Short Job First no expropiativo y Round-Robin con un quantum de 2.

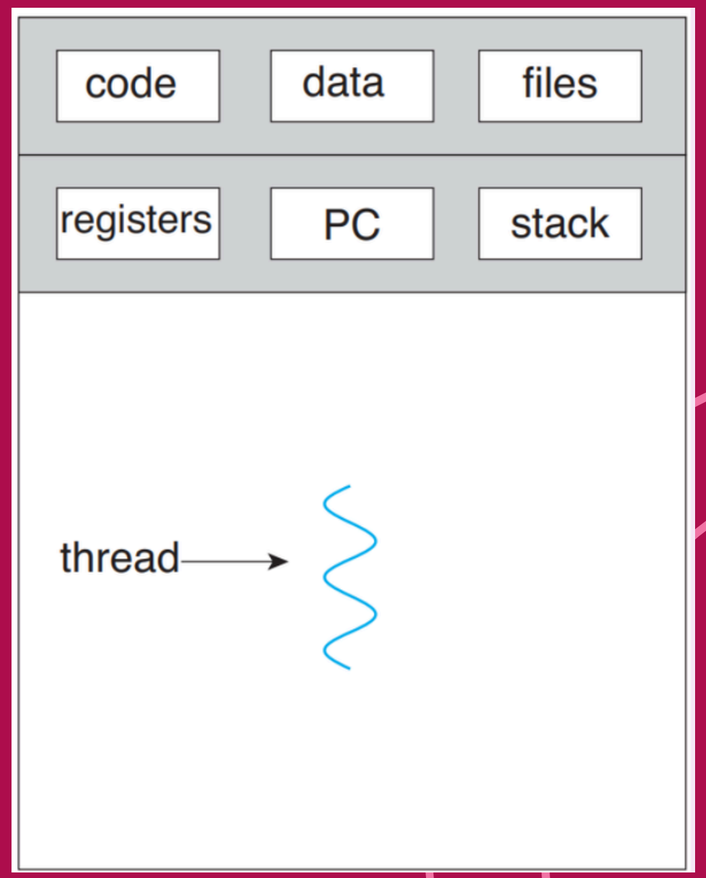


¿Qué es un thread?

Se puede decir que es como un mini-proceso dentro del un proceso (como los antes vistos), hechos para realizar tareas específicas (como una función).

Son más ligeros que realizar otro proceso nuevo, ya que comparten datos y recursos con el proceso en el que se ejecutan.





Sección Crítica

Región de la memoria de acceso compartido (variables globales), el acceso y modificación de estas variables puede cambiar el resultado del código, dando como resultado outputs que no sabemos (cosas que no queremos que sucedan jeje).

Respecto a la sección critica se busca idealmente:

- **Exclusión Mutua:** A lo más un thread/proceso en la SC.
- **Progreso:** Al menos un thread/proceso puede entrar en la SC, en caso que 2 o más quieran acceder a esta SC, se debe decidir cual en un tiempo acotado.
- **Ausencia de Inanición:** Si un thread/proceso quiere acceder a su SC, eventualmente podrá hacerlo después de un tiempo acotado.



Race Conditions

Cuando dos o más threads/procesos compiten por modificar una variable global, llevando a outputs erroneos y posiblemente imposibles de predecir.



Sincronización

¿Cómo se evita todo lo anterior? Sincronizando procesos. Opciones disponibles:

- **Desactivar las interrupciones de tu SO** (no recomendable jeje)



- **Spinlock:**

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

int lock = 0;

void *thread_func(void *arg){
    while(lock);
    lock = 1;
    //  SC
    lock = 0;
}

int main(){
    pthread_t threads[4];
    for(int i = 0 ; i < 4 ; i++){
        pthread_create(&threads[i], NULL, thread_func, NULL);
    }
    for(int i = 0 ; i < 4 ; i++){
        pthread_join(threads[i], NULL);
    }
    return 0;
}
```

Sincronización

- **Algoritmo de peterson** (Para 2 threads):

```
bandera[0] = false
bandera[1] = false
turno      // No es necesario asignar un turno
p0: bandera[0] = true                    p1: bandera[1] = true
    turno = 1                                turno = 0
    while( bandera[1] && turno == 1 );       while( bandera[0] && turno == 0 );
    //{ no hace nada; espera. }              //{ no hace nada; espera. }
    // sección crítica                       // sección crítica

    // fin de la sección crítica             // fin de la sección crítica
    bandera[0] = false                       bandera[1] = false
```

Pueden fallar si hay cambios de contexto, ya que no son atómicos, son instrucciones ejecutadas en distintas líneas, para evitar esto hay técnicas como “test_and_set” o “compare_and_swap”

Busy Waiting

Estas técnicas generan busy waiting, que en palabras simples, los threads que estén en el while esperando su turno de entrar a la SC estarán gastando tiempo de CPU también, aunque en realidad no estén haciendo absolutamente nada.



Deadlock

Dos o más threds se bloquean mutuamente, quedando atrapados para toda la eternidad esperando que el otro los desbloquee.
Se pueden dar en distintas situaciones, normalmente por un mal manejo de la sincronización para la SC.



Livelock

Livelock es más complejo que ocurra y más raro que un deadlock, en este se bloquean y **no progresan** en las instrucciones, sin embargo, si están cambiando de estado y/o realizando tareas.



Ejercicio 1

Considere la siguiente tabla de procesos:

Proceso	CPU Burst	Arrival Time
P_0	3	7
P_1	10	1
P_2	3	0
P_3	6	5

Determinar el Turnaround y end time, considerando para ello el algoritmo Short Job First no expropiativo **(15 puntos)** y Round-Robin con un quantum de 2 **(15 puntos)**.

Ejercicio 2

Tras el éxito de los profesores de SO jugando “Anillo Viejujo”, motivaron a los ayudantes Diego y Dante a realizar una nueva investigación. Esta vez, su objetivo sería analizar los diferentes órdenes posibles para derrotar a los diversos seres malvados del juego, con el fin de planificar una estrategia, ahora bajo condiciones especiales. Algunos de estos seres malvados pueden ser derrotados desde el inicio de la partida, mientras que otros requieren esperar cierto tiempo antes de poder enfrentarlos. Cada ser malvado demanda una cantidad de intentos para ser vencido (cada intento toma una unidad de tiempo) y, una vez derrotado, otorga una cantidad determinada de experiencia.

Ejercicio 2



(a) Al profesor Víctor le gusta el desafío, por lo tanto, prefiere enfrentar primero a los jefes que otorgan mayor experiencia. Por ello, encarga a su ayudante Diego realizar un intento bajo estas condiciones. ¿Qué algoritmo debería utilizar Diego en este caso? Comente qué sucede con la continuidad en el enfrentamiento de los seres malvados.

(b) El profesor Martín prefiere vencer los seres malvados en el orden en que se aparezcan, para no tener una curva de dificultad tan elevada hacia los seres malvados más complicados. Por ello, encarga a su ayudante Dante realizar un intento bajo estas condiciones. ¿Qué algoritmo debería utilizar Dante en este caso? Comente qué sucede con la continuidad en el enfrentamiento de los seres malvados.

Ejercicio 2

(c) Los ayudantes, buscando perfeccionar su técnica en el juego para enfrentar mejor a los seres malvados, deciden realizar 4 intentos consecutivos contra cada uno antes de pasar al siguiente. Para ello, disponen de la siguiente planilla de investigación:

Ser malvado	Tiempo de aparición	Intentos	Experiencia entregada
Godrick el Genio de las empanadas	0	10	80
Radrán el Guatón de la parrilla	7	8	100
Moog el príncipe del persa Bio-Bio	18	6	35
Ricarda la Reina del churrasco	24	5	60
Malena la Inalcanzable de Maipú	36	12	90
Marguito el Funado de la micro 210	12	9	75
Rennala la diosa de la cazuela	5	15	55

Completando el análisis, se pide enunciar un algoritmo para realizar esta tarea y que, para cada ser malvado, agregue a la tabla la siguiente información: tiempo de término, turnaround, tiempo de inicio del enfrentamiento con el ser malvado y demora para iniciar el enfrentamiento.

Ejercicio 3

Considere los siguientes procesos, los cuales deben ser planificados:

Proceso	Tiempo de llegada	CPU burst	Prioridad
P_1	0	20	40
P_2	25	25	30
P_3	30	25	30
P_4	60	15	35
P_5	100	10	5
P_6	105	15	10

Tabla 1: Tabla de procesos

A cada proceso le es asignado una prioridad numérica, en donde un número mayor indica una mayor prioridad. Para realizar la planificación se usa un algoritmo conocido como *Brisket-Based Round Robin (BBRR)*. Este algoritmo funciona igual que Round-Robin (con $q = 10$), pero un proceso es expropiado solamente si el siguiente proceso en la cola tiene prioridad igual o mayor que el actual en ejecución. En base a lo anterior, se pide:

1. **(10 puntos)** Mostrar el orden de ejecución (con solo una CPU) de los procesos usando una carta gantt o línea temporal.
2. **(3 puntos)** ¿Cuál es el turnaround de cada proceso?
3. **(3 puntos)** ¿Cuál es el tiempo de respuesta (*response time*) de cada proceso?
4. **(4 puntos)** Indique el porcentaje de utilización efectiva de la CPU durante toda la ejecución.
5. **(2 puntos)** Indique verdadero, falso o depende para la siguiente afirmación (basado en la Tabla 1), justificando su respuesta en no más de 3 líneas: *Aumentando la cantidad de CPUs, para todo tiempo t siempre existirá al menos un proceso que se esté ejecutando.*